Standardizing Hybrid Automata: From Theoretical Foundations to Real-World Implementation Frameworks

A Pilot Literature on the Development of a Unified Framework for Hybrid Automata Development and Simulation

Ryan McKee

Feb 2026

Abstract

Hybrid automata are formal models for dynamical systems with both discrete and continuous components [12], widely used across research and industry for applications ranging from control systems and robotics to financial algorithms and cyber-physical systems. A recent example is the modeling of COLREG-compliant autonomy for maritime vehicles [16], where hybrid automata capture both the discrete decision-making of the vessel and its continuous motion dynamics.

This theoretical framework provides a powerful method for modeling complex systems, enabling algorithmic problem-solving without human intervention, while maintaining transparency for human operators to understand the system‘s decision-making process.

Despite significant advances in artificial intelligence (AI), particularly through large-scale models like OpenAI's GPT series [14] that demonstrate human-like reasoning, challenges remain for real-world deployment. AI systems often suffer from limited transparency, explainability, and reliability, particularly in novel situations, due to the black-box nature of deep neural networks and sensitivity to outliers in training data.

Given these challenges, hybrid automata remain one of the most robust tools for complex system design. However, while the theoretical framework is well-established, there is a lack of standardization in translating it into practical applications. Different implementations use varying algorithms, making cross-comparison and adoption challenging.

In this pilot literature review, we examine the current state of hybrid automata development and simulation, identify gaps in the field, and propose a unified framework for their design and deployment. Such a framework aims to standardize the development process and enhance the efficiency, effectiveness, and applicability of hybrid automata across research and industry.

0.1 introduction

Hybrid automata are formal models for dynamical systems that combine discrete modes with continuous dynamics [12]. They have been widely adopted in both research and industry for modeling complex hybrid systems such as control systems, autonomous vehicles, and cyber-physical applications. For example, hybrid automata have been used to model decision-making and motion dynamics in maritime autonomy under COLREG rules [15]. This modeling framework provides a transparent, modular way to design and analyze systems where logic transitions and physical processes interact.

In recent years, machine learning (ML) and large AI models (e.g. GPT-4) have demonstrated remarkable capabilities on many tasks, but they often act as "black boxes" with limited interpretability. In contrast, hybrid automata yield inherently explainable models (states and transitions are explicit) and can provide formal guarantees on behavior. Consequently, hybrid automata remain a powerful tool for safety-critical and complex system design. However, despite a rich theoretical foundation, there is a lack of standardization in how hybrid automata are implemented and deployed. Different research groups and industries often develop their own modeling frameworks or code libraries, without a common API or data format. In this review, we survey the state-of-the-art in hybrid automaton development and simulation to identify gaps and motivate the need for a unified framework.

0.2 background

A hybrid automaton H can be defined as a tuple $H = (Q, X, f, Init, Inv, E, G, R)$, where Q is the set of discrete modes, X is the continuous state space, f defines the continuous dynamics in each mode, $Init$ specifies initial states, Inv assigns an invariant set to each mode, E is the set of transitions (edges) between modes, G assigns guard conditions to edges, and R assigns reset maps on transitions [12]. Informally, the system resides in some mode $q \in Q$ and evolves according to $\dot{x} = f(q, x)$ as long as the state satisfies the invariant $\mathrm{Inv}(q)$. When the continuous state reaches a guard $G(q, q')$, the automaton may take a discrete jump to mode q', resetting the continuous state via $R(q, q', x)$. This hybrid formalism generalizes finite automata by adding differential equation dynamics and state resets [12].

A simple example is a thermostat system with modes $\{On, Off\}$. The continuous variable is temperature x. In mode On, the heater is active and the temperature evolves as $\dot{x} = 5 - 0.1x$ until x reaches an upper bound (invari-

ant $x \leq 22$). In mode Off, the heater is inactive and $\dot{x} = -0.1x$ until x reaches a lower bound (invariant $x \geq 18$).

Transitions between On and Off are governed by **guards with hysteresis**: $Off \rightarrow On$ occurs when $x < 19$, and $On \rightarrow Off$ occurs when $x > 21$. No reset is applied to x on transitions (i.e., R is identity).

This example illustrates how hybrid automata capture both the discrete controller (heater on/off) and the continuous physics (temperature dynamics) in a single model, visualized in figure 1.

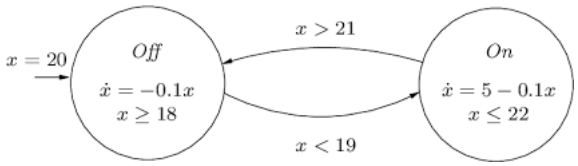

Figure 1: Relating Chi to hybrid automata – Scientific Figure on ResearchGate. Available from: `https://www.researchgate.net/figure/A-Hybrid-Automaton-Model-of-a-Thermostat_fig1_4053758` [accessed 18 Feb 2026]

0.3 State of the art

A variety of specialized tools and frameworks exist for modeling, verifying, and simulating hybrid systems. Formal analysis tools for hybrid automata include **SpaceEx** [11] for reachability of piecewise-linear dynamics, **C2E2** [9] for bounded simulation-based safety checking of nonlinear hybrid models, and **Flow*** [8] for computing over-approximations of reachable sets using Taylor models. Tools like **CORA** [4] provide MATLAB-based reachability for linear systems, and **PHAVer** [10] handles exact verification for rectangular hybrid automata. Many of these tools require models in their own input language or have steep learning curves. For example, SpaceEx uses an XML input format for hybrid automata [11], and C2E2 expects annotated ODE models [9]. Efforts such as the **HYST** translator allow a model to be converted among several tools [5], but no universally adopted modeling language has emerged.

There are also hybrid modeling libraries and simulators. For instance, HyAuLib provides Modelica components for building hybrid automata in the Modelica environment [2]. The Hybrid Automata Library (HAL) [7] is a Java library of generic components (agent containers, PDE fields, etc.) to facilitate hybrid

spatial models in oncology. MATLAB/Simulink users can employ toolboxes (e.g. the Hybrid Toolbox by Bemporad et al. [6]) to design and simulate hybrid controllers, though these often target control design rather than general hybrid automaton semantics.

In the Python ecosystem, recent work has begun to address hybrid systems. The **Verse** library [13] provides a Python framework for multi-agent hybrid scenarios, where each agent's controller is written in Python and dynamics are given by black-box simulators. VERSE allows simulation and integration with reachability backends, lowering the barrier for scenario-based verification. Other projects (e.g. Ariadne for CPS analysis [1]) offer Python bindings for verification. However, these are mostly research prototypes or focused on verification, not a general-purpose API.

Overall, the IEEE Control Systems Society's hybrid systems tool registry [3] lists dozens of tools (reachability analyzers, theorem provers, simulation libraries, etc.). While this ecosystem is rich, it is highly fragmented. Each tool often has its own modeling format or programming environment, and many are not user-friendly for newcomers. There is currently no single, standardized framework or language for defining and running hybrid automata in a lightweight way across domains.

0.4 Discussion / Reserach Gaps

Despite the wealth of tools, significant gaps remain in translating hybrid automaton theory into practice. First, there is no standard modeling language or API for hybrid automata. Different tools use different input formats (XML, MATLAB classes, custom DSLs), which makes it difficult to reuse models across tools. Projects such as **HYST** [5] have attempted to create common interchange formats, but such efforts require broad community adoption. In many cases, converting industrial models (e.g. Simulink/Stateflow) to formal hybrid automata is cumbersome and error-prone [9, 5].

Second, existing tools often focus on analysis rather than ease of integration. For example, **SpaceEx** [11] is powerful for verification of affine systems, but it is not designed as a development library or API for embedding in larger software stacks. Similarly, model-based design tools such as Simulink are tied to proprietary platforms. Few tools provide both real-time simulation and offline analysis capabilities in a lightweight and unified package.

Third, there is limited support for iterative design and deployment workflows. Researchers frequently implement custom hybrid automata for each

project, often in Python or C++, without reusing existing libraries. The lack of a common framework leads to duplication of effort and steep learning curves. In summary, hybrid systems practitioners often face a gap between elegant formal definitions and the practical code required to simulate and deploy automata in real-world applications. This gap motivates the need for a unified, standardized development framework.

0.5 Conclusions and Future Work

Hybrid automata are theoretically mature and mathematically rigorous, yet their practical ecosystem remains fragmented. Existing tools provide powerful analysis capabilities, but they differ widely in modeling languages, input formats, and integration workflows. This fragmentation creates a gap between formal hybrid automaton theory and practical deployment in real-world systems.

To address this gap, we propose the development of a unified hybrid automaton framework guided by the following principles:

- **Cross-platform, high-level API:** A Python-based library that enables users to define hybrid automata declaratively using structured data representations or a lightweight domain-specific syntax. The API should uniformly support mode definitions, invariants, guards, reset maps, and continuous dynamics while remaining intuitive and extensible.
- **Simulation and real-time execution:** Support for both offline simulation (for analysis and testing) and real-time execution suitable for embedded systems and robotic platforms. Integration with middleware such as ROS2 would enable seamless transition from simulation to deployment.
- **Visualization and tooling:** Built-in capabilities for visualizing automaton graphs, plotting continuous trajectories, and logging state transitions during execution. Such tooling would enhance interpretability and debugging.
- **Performance and modularity:** A lightweight and modular core architecture allowing users to integrate different numerical solvers or reachability backends without altering the primary API. This ensures flexibility while preserving architectural consistency.
- **Compatibility with formal verification backends:** Although pri-

marily focused on simulation and deployment, the framework should support exporting models to formal analysis tools such as SpaceEx [11] or interfacing with verification engines to enable safety analysis.

A unified framework of this nature would bridge the divide between elegant formal definitions and practical implementation. By standardizing how hybrid automata are specified, executed, and analyzed, it would promote model reuse, reproducibility, and cross-domain collaboration. Such infrastructure would support applications ranging from simple control systems to complex autonomous platforms without requiring developers to reconstruct tooling for each new project.

Future work will focus on designing the modeling language and API specification, prototyping the core architecture in Python, and evaluating the framework on benchmark hybrid systems to assess scalability, usability, and interoperability with existing verification tools.

Bibliography

[1] Ariadne: A toolbox for hybrid system analysis.

[2] Hyaulib: A modelica library for hybrid automata.

[3] Ieee control systems society hybrid systems tool registry.

[4] Matthias Althoff. *Reachability Analysis and its Application to the Safety Assessment of Autonomous Cars*. PhD thesis, Technische Universität München, 2010.

[5] Stanley Bak, Sergiy Bogomolov, and Taylor T. Johnson. Hyst: A source transformation and translation tool for hybrid automaton models. In *Proceedings of HSCC*, 2015.

[6] Alberto Bemporad. Hybrid toolbox for matlab, 2005.

[7] Ricardo Bravo and Alexander R. A. Anderson. Hybrid automata library: A flexible platform for hybrid modeling. *PLoS Computational Biology*, 2019.

[8] Xin Chen, Erika Ábrahám, and Sriram Sankaranarayanan. Flow*: An analyzer for nonlinear hybrid systems. In *Proceedings of CAV*, 2013.

[9] Chuchu Fan, Bolun Qi, and Sayan Mitra. C2e2: A verification tool for stateflow models. In *Proceedings of TACAS*, 2016.

[10] Goran Frehse. Phaver: Algorithmic verification of hybrid systems. In *Proceedings of HSCC*, 2007.

[11] Goran Frehse, Colas Le Guernic, Alexandre Donzé, Scott Cotton, Radu Ray, Olivier Lebeltel, Rodolfo Ripado, Alexandre Girard, Thao Dang, and Oded Maler. Spaceex: Scalable verification of hybrid systems. In *Proceedings of CAV*, 2011.

[12] T.A. Henzinger. The theory of hybrid automata. In *Proceedings 11th Annual IEEE Symposium on Logic in Computer Science*, pages 278–292, 1996.

[13] Yilun Li et al. Verse: A framework for verification of multi-agent hybrid systems. In *Proceedings of CAV*, 2023.

[14] OpenAI. Gpt-4 technical report, 2024.

[15] International Maritime Organization. https://www.imo.org/en/about/conventions/pages/co 2024.

[16] Bhawana Singh, Karim Ahmadi Dastgerdi, Nikolaos Athanasopoulos, Wasif Naeem, and Benoit Lecallard. Safe colregs-aware finite-time guidance and control of marine vehicles. *Ocean Engineering*, 351:123919, 2026.